iscte TECNOLOGIAS E ARQUITETURA

Software Engineering

Software Construction with Reuse (Build Automation Tools)

Overview

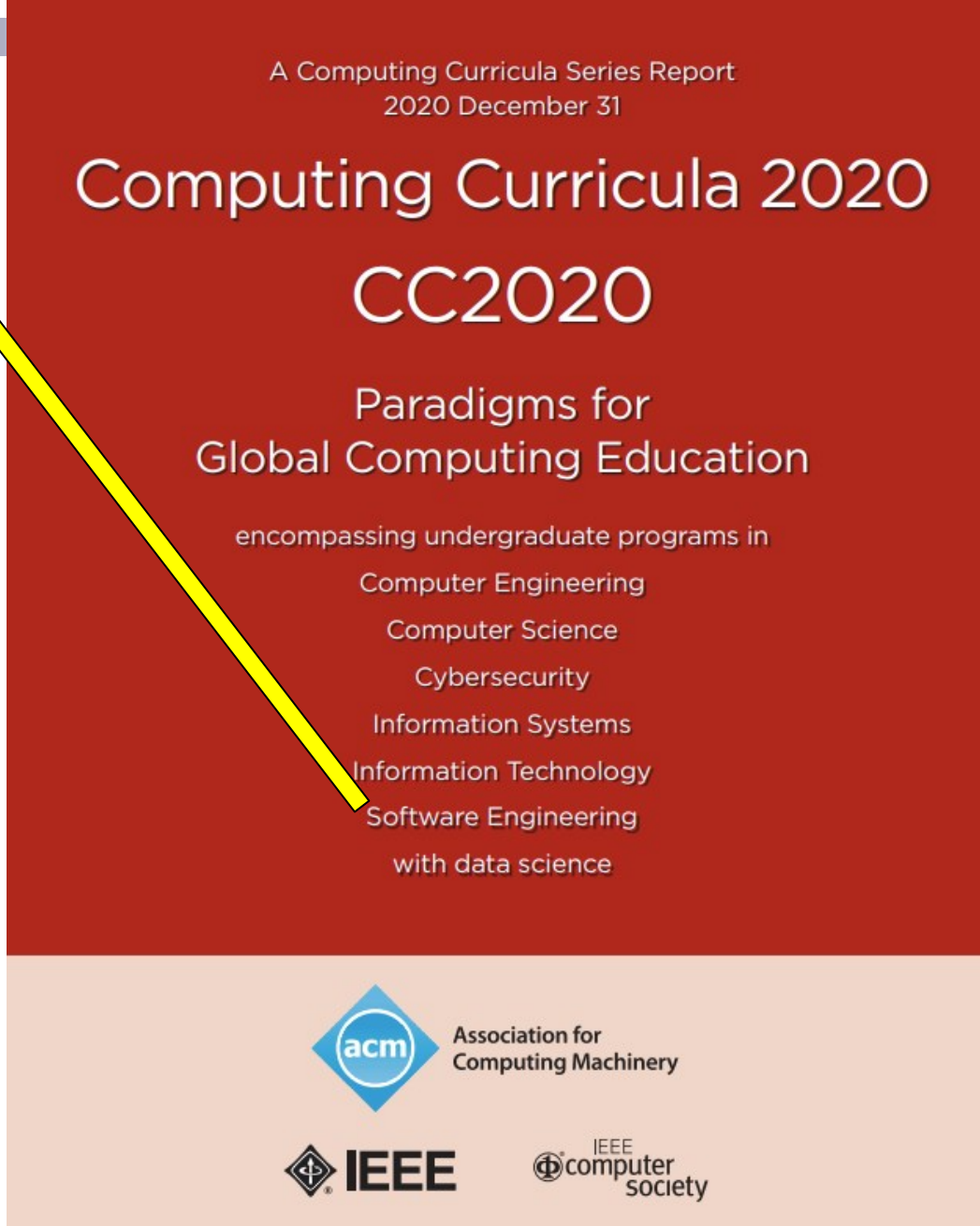
- Motivation
- Construction for Reuse
  - Reusable assets repositories
  - Repositories' characteristics
- Construction with Reuse
  - Package Dependencies
  - Vulnerabilities
  - Package Managers
- Build Automation Tools
  - Characteristics
  - Market share
- Maven
  - Project Object Model (POM) file
  - SNAPSHOT versions
  - Package management
  - Repositories
  - Project Templates/Archetypes
  - Lifecycles, Plugins, Phases and Goals

Soft. Eng. Competences

- Software Requirements
- Software Design
- Software Construction
- Software Testing
- Software Sustainment
- Software Process and Life Cycle
- Software Systems Engineering
- Software Quality
- Software Security
- Software Safety
- Software Configuration Manag.
- Software Measurement
- Human-Computer Interaction
- Project Management
- Behavioral Attributes

CDSI

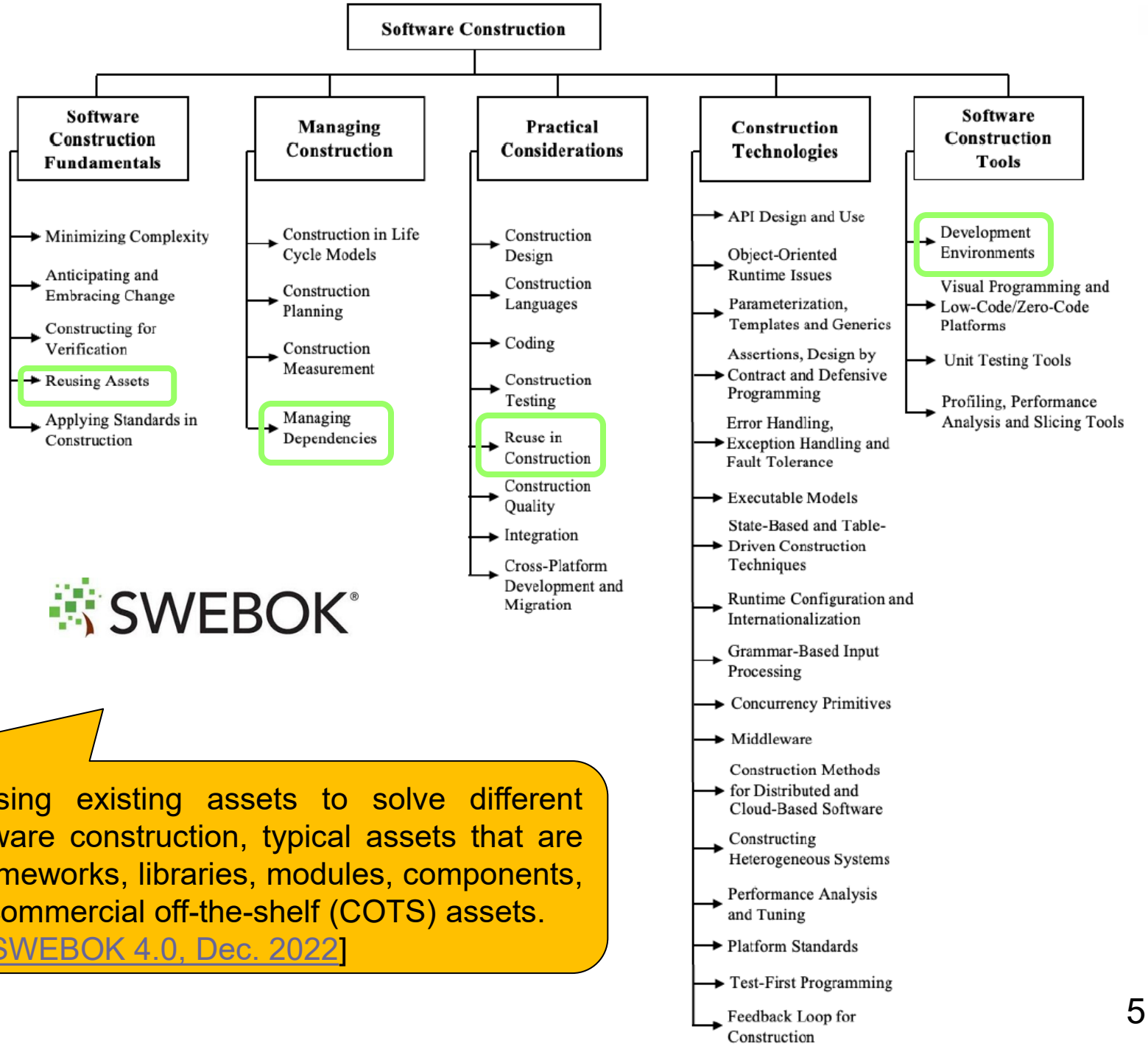
IPM



Software Engineering Body of Knowledge

1. Software Requirements
2. Software Architecture
3. Software Design
4. Software Construction
5. Software Testing
6. Software Engineering Operations
7. Software Maintenance
8. Software Configuration Management
9. Software Engineering Management
10. Software Engineering Models and Methods
11. Software Engineering Process
12. Software Quality
13. Software Security
14. Software Engineering Economics
15. Software Engineering Professional Practice
16. Computing Foundations
17. Mathematical Foundations
18. Engineering Foundations

Refers to the detailed creation and maintenance of software through coding, verification, unit testing, integration testing and debugging. [SWEBOK 4.0, Dec. 2022]



SWEBOK®

Reuse means using existing assets to solve different problems. In software construction, typical assets that are reused include frameworks, libraries, modules, components, source code and commercial off-the-shelf (COTS) assets.
[SWEBOK 4.0, Dec. 2022]

Construction for reuse (assets repositories)



































ComponentSource raised $19.8M in funding over 2 rounds

Construction for reuse (assets repositories)

	DOCUM. MANAGEM.	RETAIL	TOURISM	HEALTH	BANKING	ALL AREAS
REPOSITORY	OCR	BarCode	QRcode	Image Processing	Exchange Rates	Calendar
MVN Central Repository	ND	ND	ND	ND	ND	ND
npm	D	D	D	D	D	D
NuGet	D	D	D	ND	ND	D
pub.dev	D	D	D	D	D	D
RubyGems	D	D	D	D	D	D
Component Source	ND	D	ND	ND	ND	D

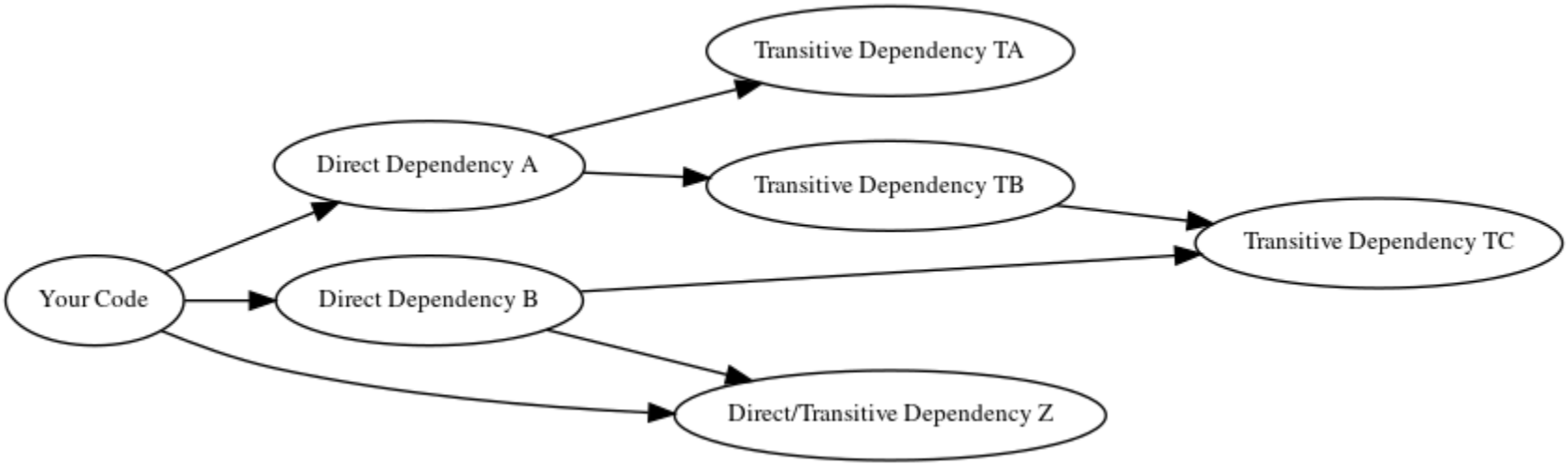
D = Documented
ND = Not Documented

Construction for reuse (assets repositories)

REPOSITORY	Dimension (# assets)	Organization / Search	Metadata (GAV)	Document.	Vulnerab. reporting	Usage reporting
MVN Central Repository	10,7M	Multicriteria	Yes	No	Yes	Yes
npm	N/A	Monocriteria	Yes	Yes	No	Yes
NuGet	344K	Multicriteria	Yes	**Yes**/No	No	Yes
pub.dev	33K	Multicriteria	Yes	Yes	Yes	Yes
RubyGems	175K	Multicriteria	Yes	Yes	No	Yes
Component Source	2K	Multicriteria	Yes	Yes/**No**	No	Yes

Construction with reuse (package dependencies)

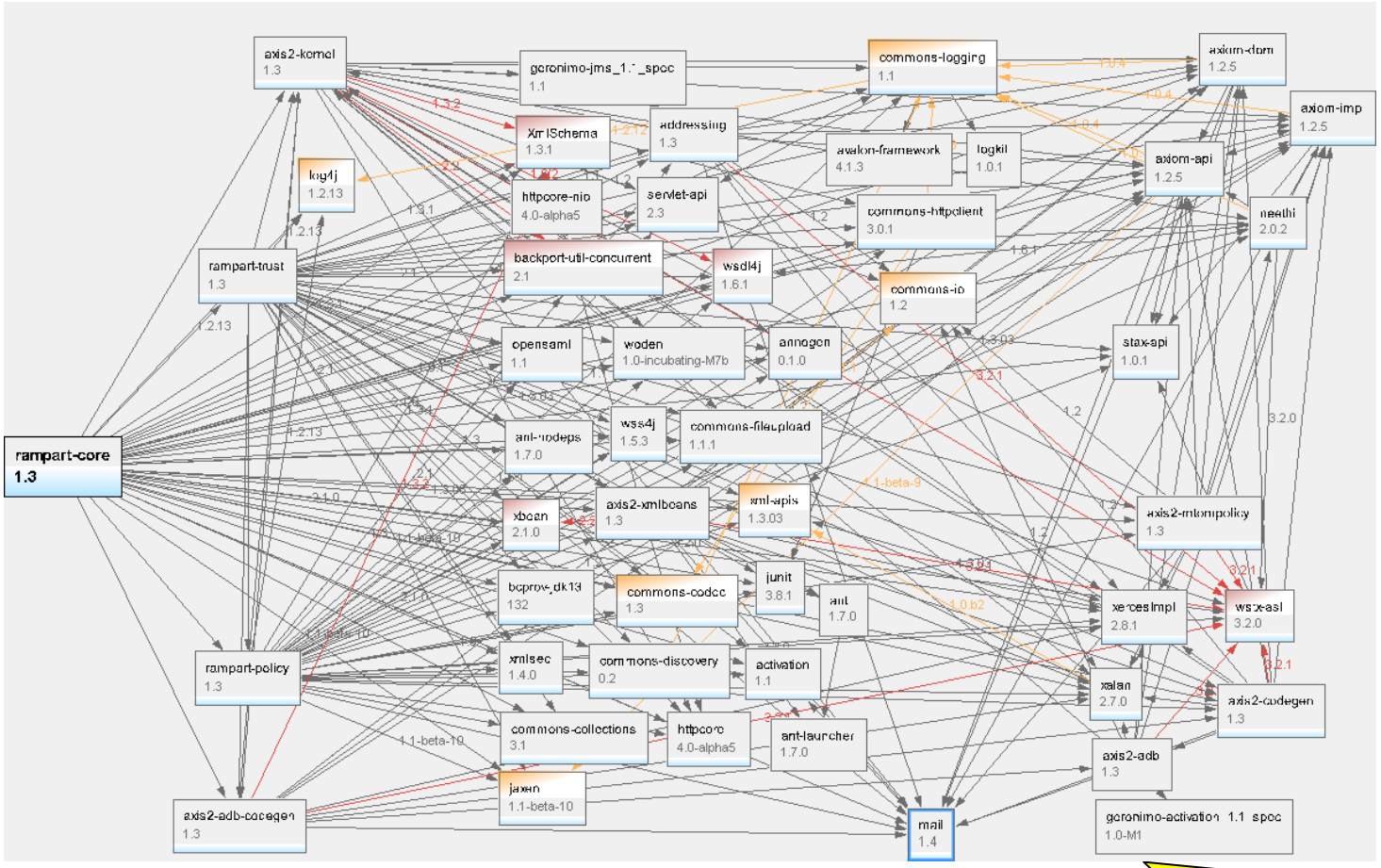
"**Package**" is a usual name for a distributable (reusable) software asset, such as a library, component or framework. Consider the ellipsis are packages and arrows are dependencies among them.



Only **direct dependencies** are known by the producer of each package, but **transitive (indirect) dependencies** often exist. The problem is that corresponding binaries / byte code should be available at runtime!

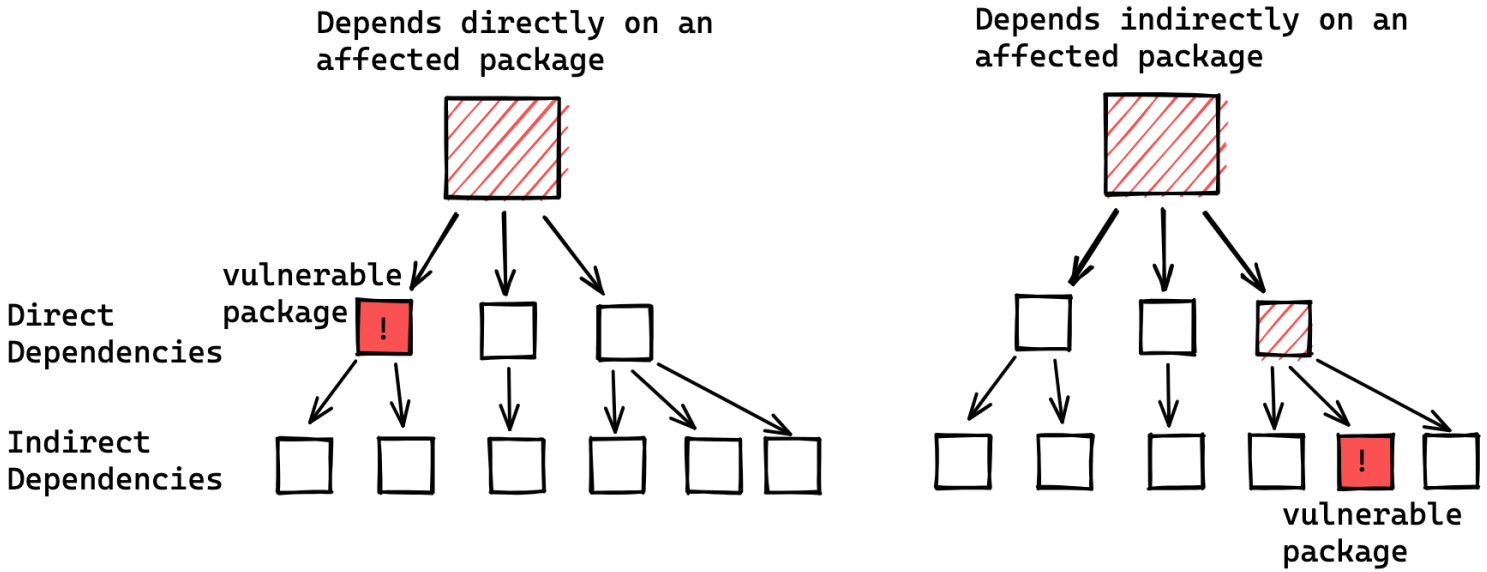
Construction with reuse (package dependencies)

Package dependencies may be vey complex



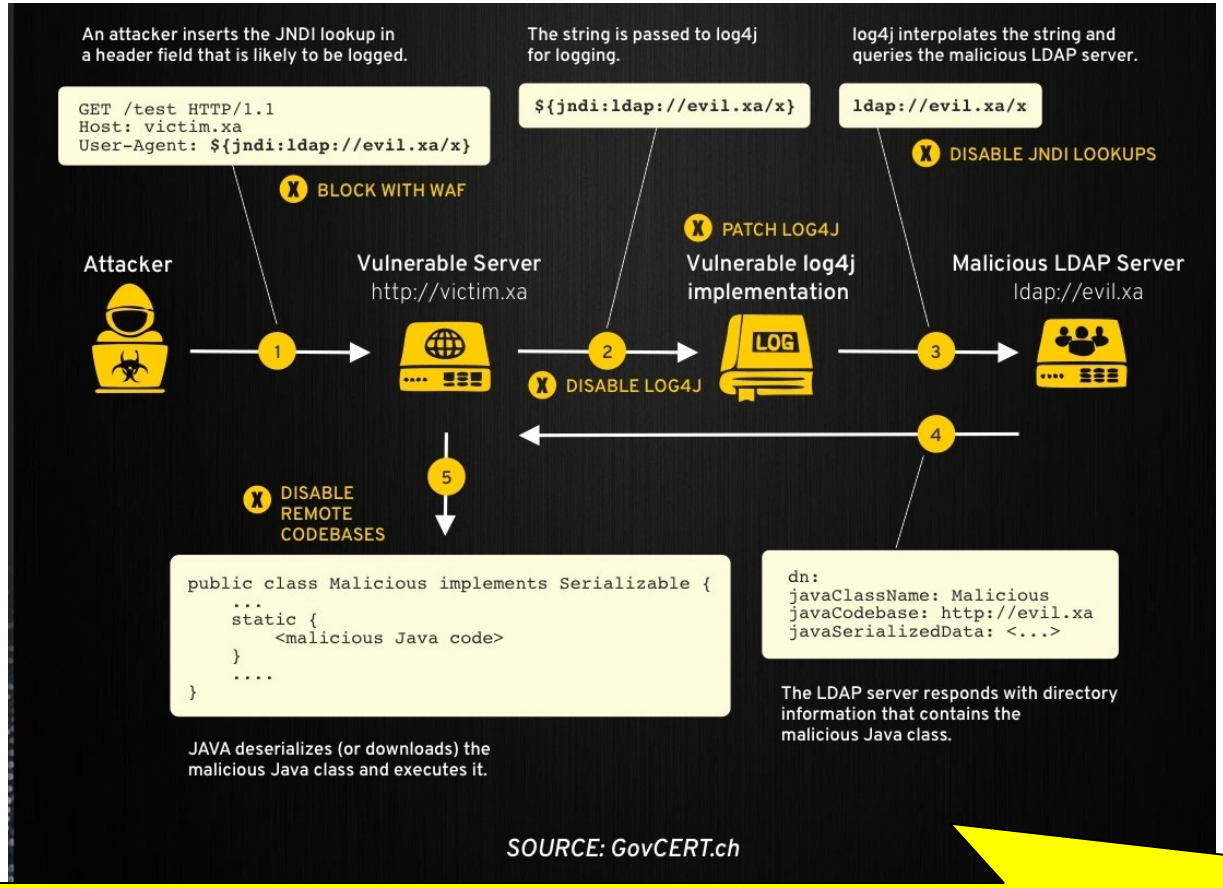
Tools exist for drawing dependency graphs which are useful for checking transitive vulnerabilities

Construction with reuse (vulnerabilities)



Note: in Java, packages are stored in repositories as JAR (Java ARchive) files

Construction with reuse (vulnerabilities)



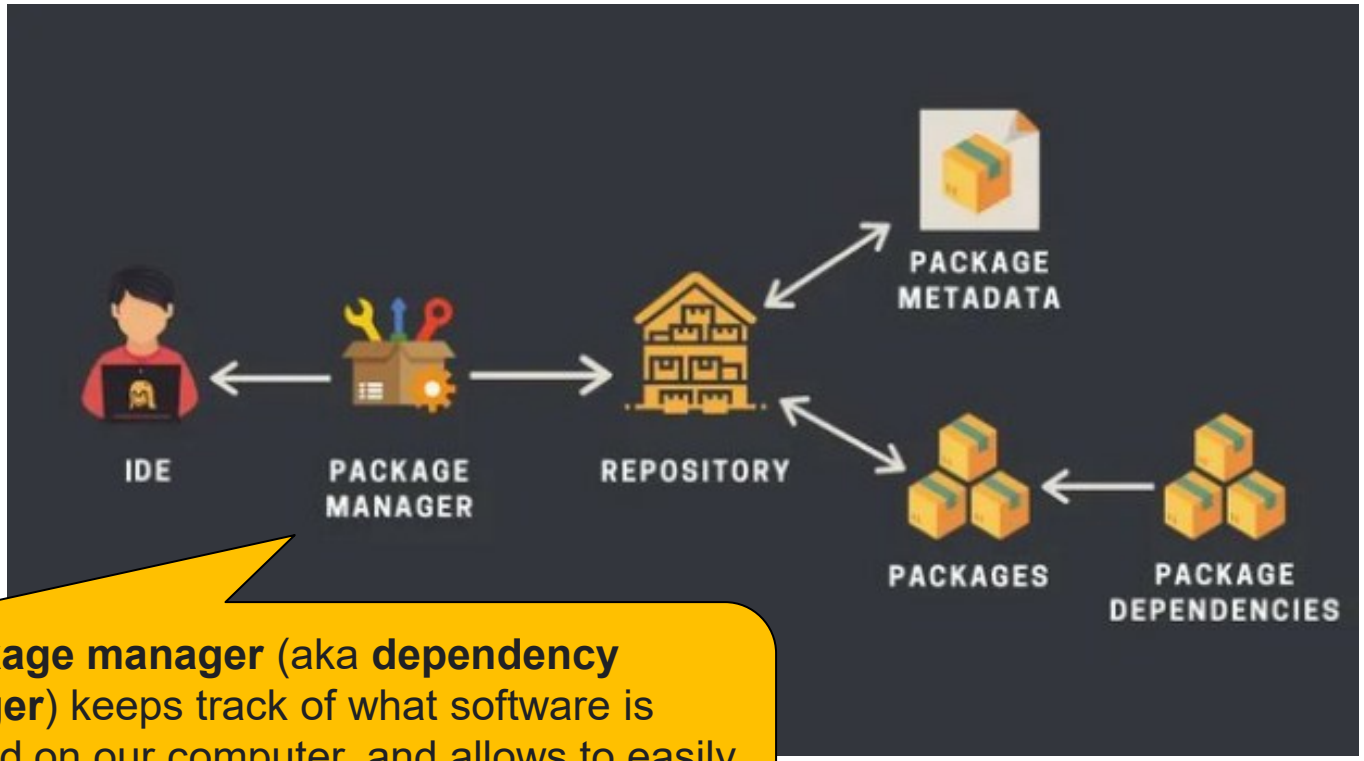
See how many vulnerabilities here!



Apache Log4j (logging library) vulnerability [December 2021]

- Allows an attacker to perform remote code execution by exploiting the insecure JNDI (*Java Naming and Directory Interface*, included in Java SE Platform) API lookups feature exposed by log4j. This exploitable feature was enabled by default in many versions of the library.
- More than 17,000 Java packages (over 4% of the MVN Central repository were impacted.

Construction with reuse (package manager)



A **package manager** (aka **dependency manager**) keeps track of what software is installed on our computer, and allows to easily install new software, upgrade it to newer versions, or remove previously installed ones.

Construction with reuse (package managers)

Language	Package managers
Java  Kotlin 	Maven, Gradle, Ivy
Groovy 	Grape
Clojure 	Leiningen
Python 	pip
JavaScript 	npm
.NET 	*NuGet*
Flutter/Dart 	pub
Ruby 	gem
PHP 	Saeghe
Haskell 	cabal

Specification of package dependencies



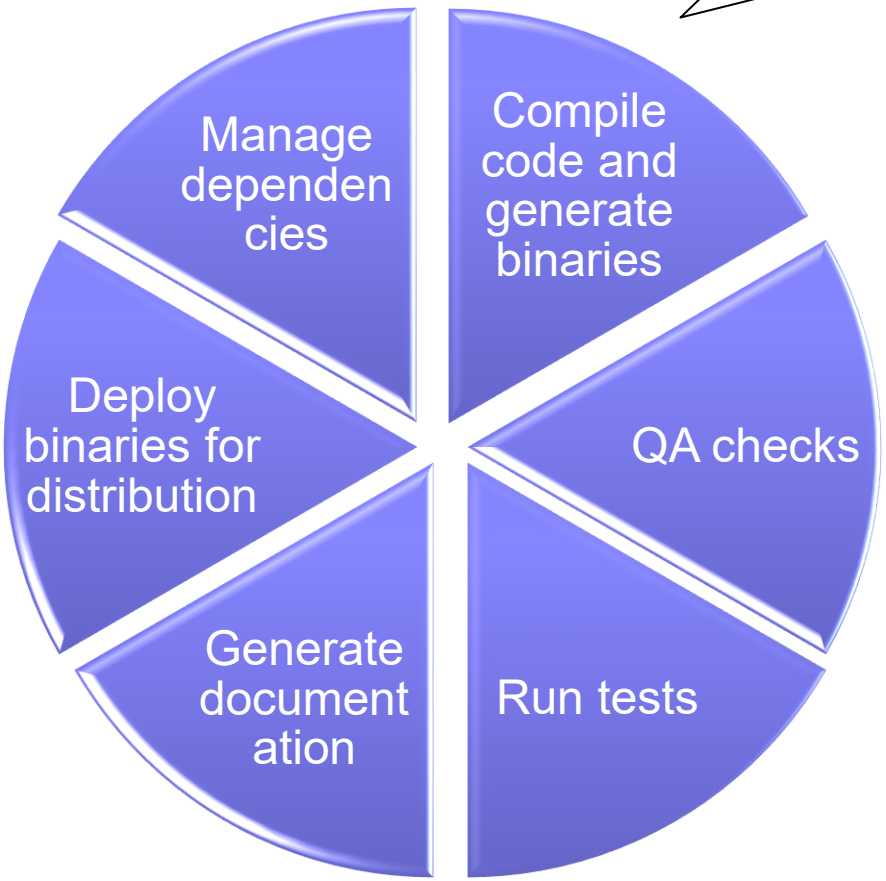
Manager	Dependency declaration
Maven	<dependency> <groupId>org.apache.logging.log4j</groupId> <artifactId>log4j-core</artifactId> <version>2.20.0</version> </dependency>
Gradle	implementation group: 'org.apache.logging.log4j', name: 'log4j-core', version: '2.20.0'
Ivy	<dependency org="org.apache.logging.log4j" name="log4j-core" rev="2.20.0"/>
Grape	@Grapes(@Grab(group='org.apache.logging.log4j', module='log4j-core', version='2.20.0'))
Leiningen	[org.apache.logging.log4j/log4j-core "2.20.0"]

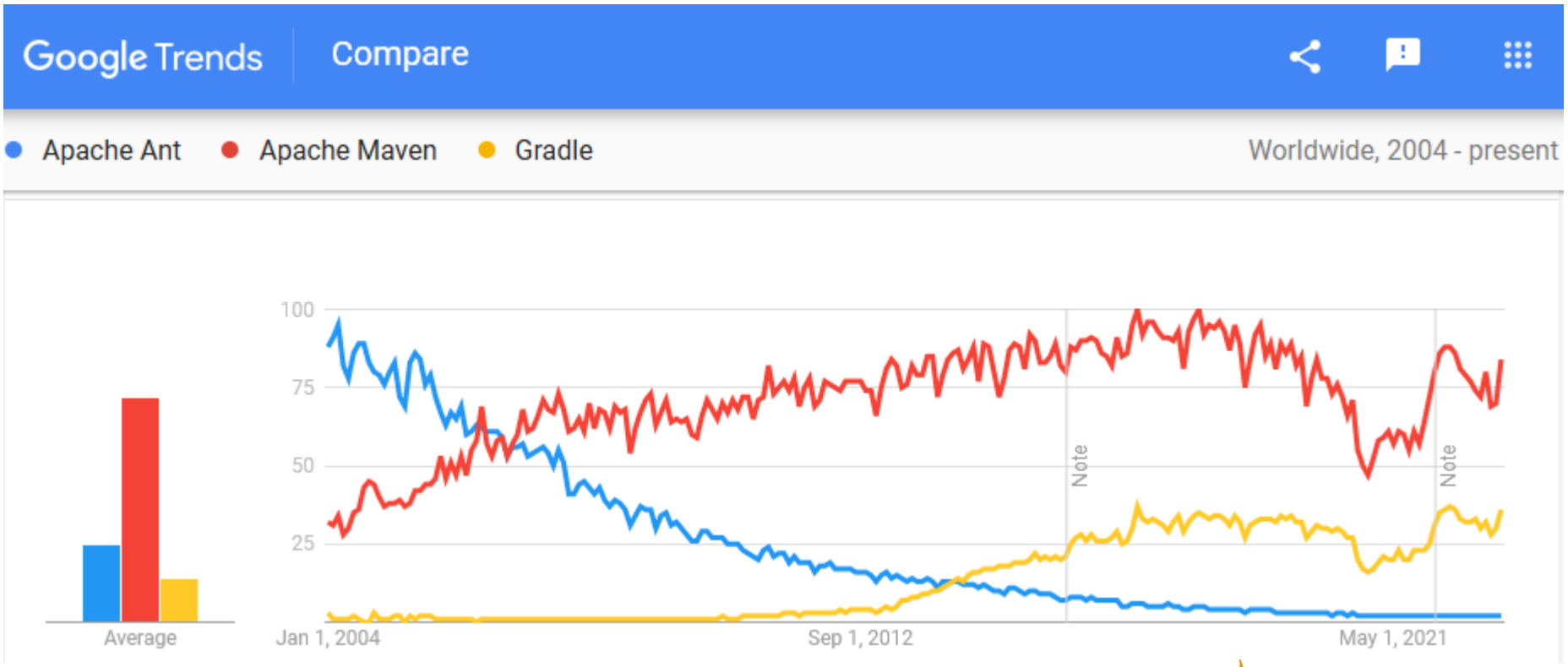
GAV (Group, Artifact, Version) is the commonality among all declarations:

- **group** is used to identify who produced the package (e.g., a company, a department or project team within a company, a research group in university) and is usually written as a reversed URL
- **artifact** stands for the name of the package (e.g., library, component, module)

Build Automation







Most used build tools for Java

Apache Maven™

- Is the build automation tool we have adopted in this course
- Provides support to automate, simplify and standardize the project build process (complete build lifecycle), namely:
  - Checks, Tests, Documentation, Dependencies, Compilation, Deployment and Team collaboration
- Uses a XML project configuration file (**pom.xml**)
- Relies on the ***principle of convention over configuration***
- Has CLI and plugins in major IDEs

```
$ mvn compile
$ mvn test
```

POM (Project Object Model) file

```
<project xmlns=...>
  <modelVersion>4.0.0</modelVersion>

  <groupId>pt.iscte-iul.istar</groupId>
  <artifactId>MiniTrader</artifactId>
  <version>0.0.1-SNAPSHOT</version>

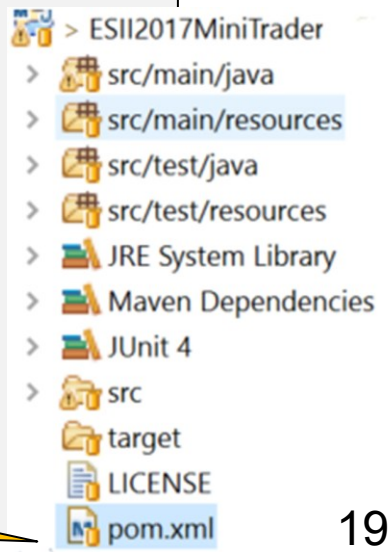
  <properties>
    <maven.compiler.source>1.7</maven.compiler.source>
    <maven.compiler.target>1.7</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.12</version>
    </dependency>
  </dependencies>

</project>
```

GAV

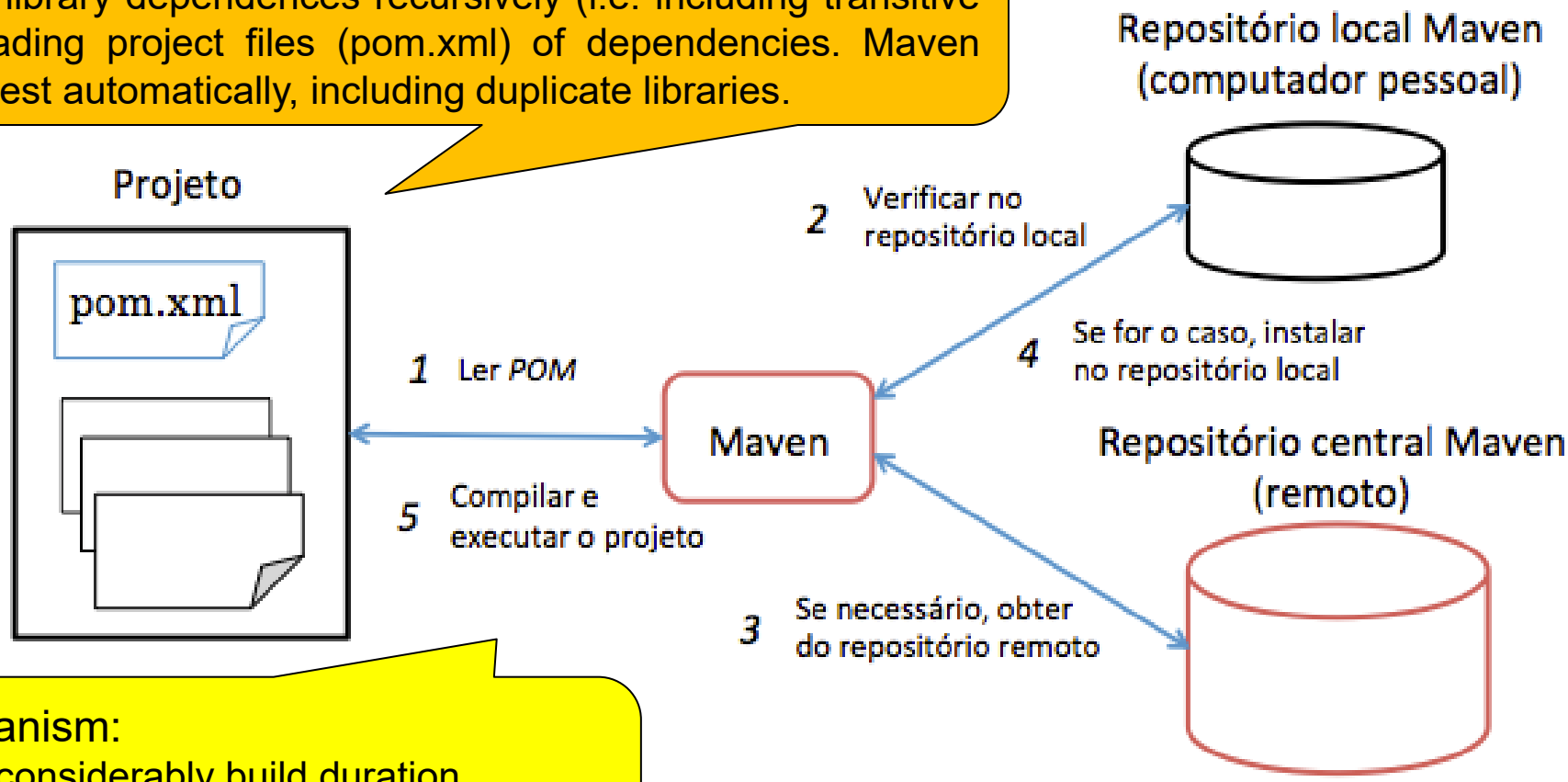
- **project**, root element
- **Basic schema settings**, XML document structure related data
- **modelVersion**, should be 4.0.0 (currently the only supported POM version for Maven 2 and 3 ; 4.1 for Maven 4)



Only one POM file per project

POM – How it works?

Only direct dependencies in each project pom are required. Maven discovers all library dependences recursively (i.e. including transitive ones) by reading project files (pom.xml) of dependencies. Maven handles the rest automatically, including duplicate libraries.



This mechanism:

- reduces considerably build duration
- eliminates the need for updating transitive dependencies (that can be many)

Repositories

Maven **build looks for dependencies in this sequence**

Repositories **can be of 3 types**:

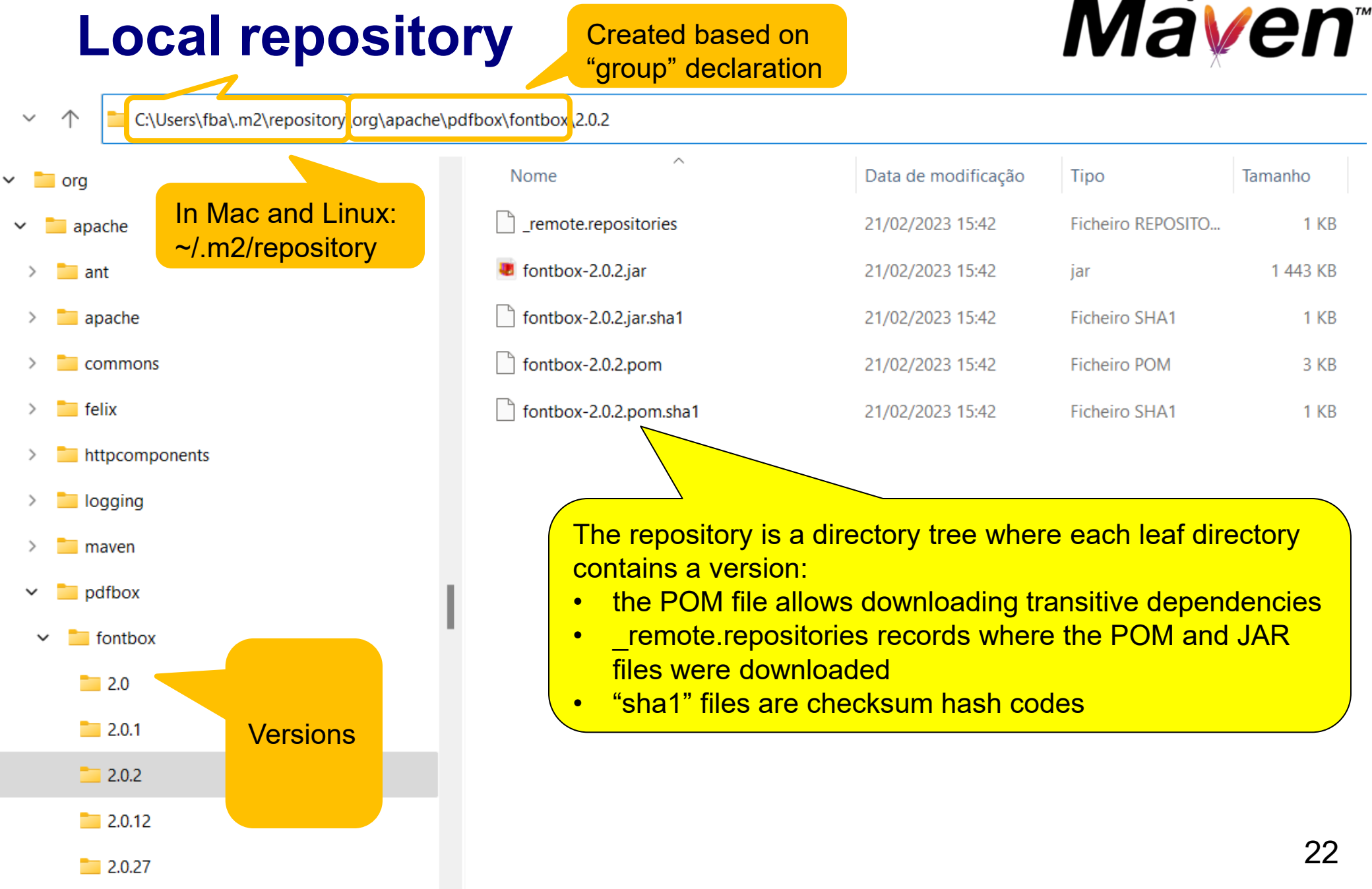
- **Local**, folder on developers' machine (%USER_HOME% directory), created the first time a Maven command is run. Maven downloads all dependencies (direct and transitive) there, during the build process.

- **Central**, managed by Maven community, contains most used libraries (tenths of millions nowadays)

MVN REPOSITORY

- **Remote**, developer's or company's own repository; are explicitly in the pom.xml file

Local repository



The repository is a directory tree where each leaf directory contains a version:

- the POM file allows downloading transitive dependencies
- _remote.repositories records where the POM and JAR files were downloaded
- "sha1" files are checksum hash codes

Remote repositories

When using specific libraries in the project, not available in Maven repositories, is possible to configure/add "External Dependency" to pom.xml in similar way to other dependencies (groupId, and artifactId should be the same as the name of the library, scope as system and system path relative to the project location):

```
<dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <dependency>
      <groupId>specificlibrary</groupId>
      <artifactId>specificlibrary </artifactId>
      <scope>system</scope>
      <version>1.0</version>
      <systemPath>${basedir}\src\lib\ specificlibrary.jar</systemPath>
    </dependency>
</dependencies>
```

```
<repositories>
  <repository>
    <id>my-repo1</id>
    <name>your custom repo</name>
    <url>http://jarsm2.dyndns.dk</url>
  </repository>
  <repository>
    <id>my-repo2</id>
    <name>your custom repo</name>
    <url>http://jarsm2.dyndns.dk</url>
  </repository>
</repositories>
```

Maven SNAPSHOT

Large applications consist of several modules developed by different teams.

- Each module might have new versions released to the remote repository, requiring other teams to update their pom.xml regularly to get the updated versions.

SNAPSHOT is a special version **indicating a development copy.**

- Maven automatically fetches the latest SNAPSHOT version in a remote repository, even if it is available in the local repository. By default, does this on a daily basis, but that is configurable.
- SNAPSHOT of updated code might be released every time to repository (e.g. service: 1.0-SNAPSHOT), replacing an older SNAPSHOT jar.

Maven SNAPSHOT

Service/project providing 1.0-SNAPSHOT service pom.xml:

```
<project ...>
    <groupId>data-service</groupId>
    <artifactId>service</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>
    ...
</project>
```

Construction **for** reuse

Application/project consuming 1.0-SNAPSHOT service pom.xml:

```
<dependencies>
    <dependency>
        <groupId>data-service</groupId>
        <artifactId>service</artifactId>
        <version>1.0-SNAPSHOT</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Construction **with** reuse

Principle of Convention over Configuration

Convention over configuration is a software design paradigm used by software frameworks to decrease the number of decisions without losing flexibility.

- A developer **only needs to specify unconventional aspects** of the application. When the convention implemented matches the desired behavior, it's not needed to write configuration files.

- Only when the desired behavior deviates from the implemented convention is explicit configuration required.

Maven Project Templates/Archetypes

Maven provides a **variety of project types/templates named Archetypes** (Maven plugins to create quickly new projects of specific types/structures).

Developers provide 'groupId', 'artifactId', 'version', 'package' and project type, **Maven generates the pom.xml and sample java source files for the project**:

- ***maven-archetype-quickstart***, contains a sample Maven project. Maven adds a project source file App.java and a test file AppTest.java in the directory structure, and Junit framework as the test framework
- ***maven-archetype-simple***, contains a simple Maven project
- ***maven-archetype-webapp***, contains a sample Maven Webapp project
- ***maven-archetype-j2ee-simple***, contains a simplifed sample J2EE application

Maven Lifecycles, Plugins, Phases and Goals

Maven is **based** on the concept of a **build lifecycle** - sequentially executed building phases.

There are a few **built-in build lifecycles** in Maven: **Default lifecycle** (handles project deployment), **Clean lifecycle**, handles project cleaning, **Site lifecycle**, handles the creation of project's site documentation.

The default lifecycle comprises the following phases:

- **validate**, validates if the project is correct and all necessary information is available;
- **compile**, compile the source code of the project;
- **test**, test the compiled source code using a testing framework;
- **package**, takes the compiled code and packages it in its distributable format, JAR, WAR, etc.
- **verify**, run any checks on results of integration tests to ensure quality criteria are met;
- **install**, install the package in the local repository, for use as dependency in other projects;
- **deploy**, done in the build environment, copies the final package to the remote repository for sharing with other developers and projects.

When a phase is called (e.g. mvn compile), only phases up to (including) that phase will execute.

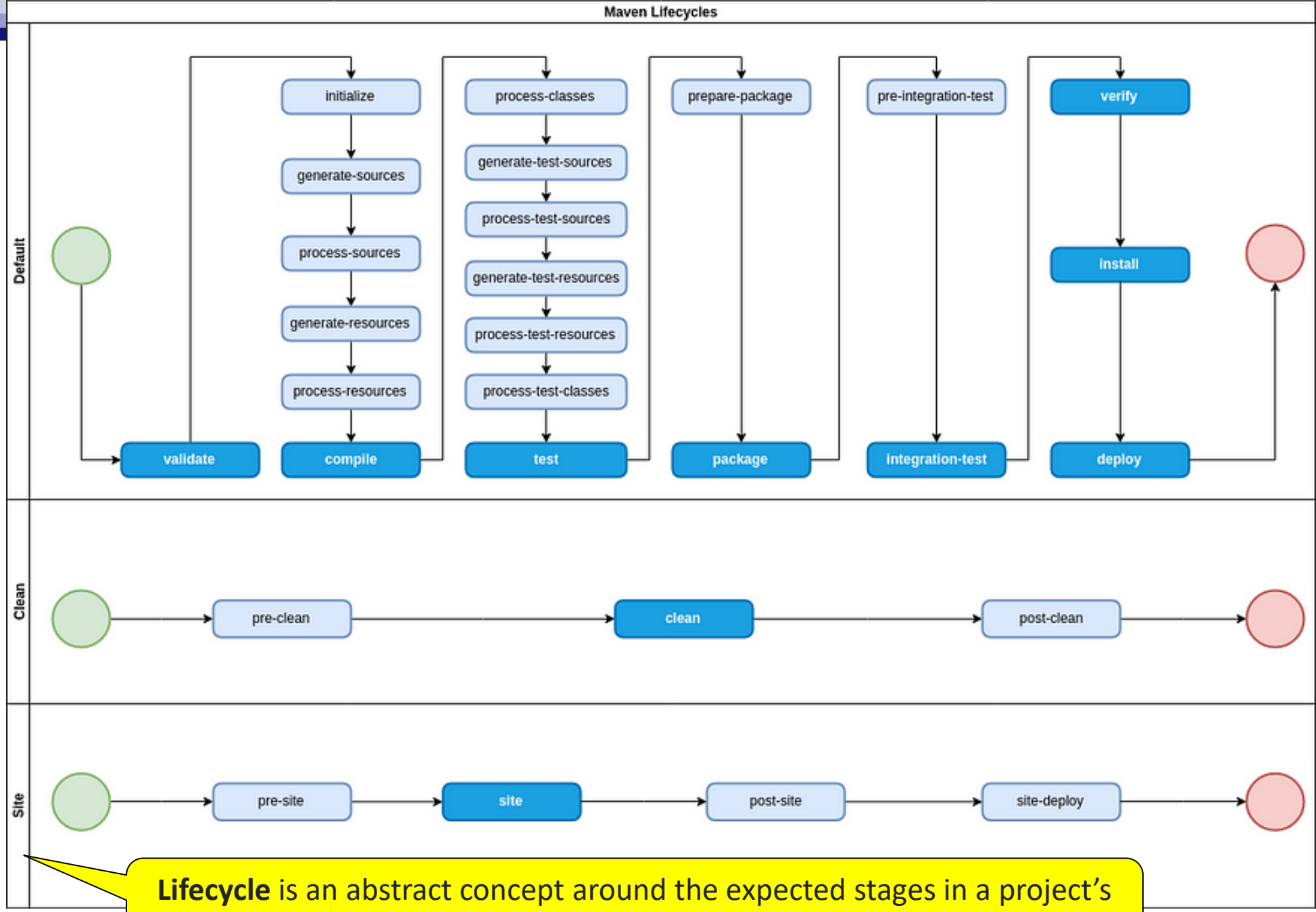
Maven Lifecycles, Plugins, Phases and Goals

- Maven is a **plugin execution framework**, where tasks are done by plugins;
- Plugins provide a set of goals which are tasks finer than build phases;
- A plugin may provide a set of goals, which can be executed using the syntax: mvn [plugin-name]:[goal-name]
- The way a building phase is performed can vary by declaring/bounding plugin goals to that phase (a goal can be bound to different phases);
- Maven lifecycle with different types of packaging (jar, war, etc.) have different goals bound to different phases of the lifecycle.

Maven Plugins

List of common plugins:

- clean, cleans up target after the build, deletes the target directory
- compiler, compiles Java source files
- surefire, runs the JUnit tests and creates test reports
- jar, builds a JAR file from the current project
- war, builds a WAR file from the current project
- Javadoc, generates Javadoc for the project



Lifecycle is an abstract concept around the expected stages in a project's development lifetime. It is a sequence of named stages i.e **phases.**